

Humboldt-Universität zu Berlin
Einführung in wissenschaftliches Rechnen
Aufgabenblatt 5
Gruppe 08

Bericht

Bestimmung aller Primzahlen kleiner gleich $n \in \mathbb{N}$

Mai Nguyen, Ele Tarielasvhili

Abgabedatum 04. Juli 2024

Betreut von
Prof. Hella Rabus

Inhaltsverzeichnis

1	Einführung und Motivation	3
2	Theorie	3
2.1	Naiver Algorithmus	4
2.2	Sieb des Eratosthenes	5
2.3	Sieb von Atkin	5
2.4	Komplexität von Algorithmen	7
3	Experimente	9
3.1	Laufzeitmessungen	10
3.2	Speicherbedarfsmessungen	12
4	Auswertung	13
5	Zusammenfassung	13
6	Python-Dokumentation	15
6.1	Bedienungsanleitung und Hauptprogramm	15
6.1.1	Bedienungsanleitung	15
6.1.2	Beispielausführung des Hauptprogramms	16
6.2	Schnittstellendokumentation	18
6.2.1	algorithmen.py	18
6.2.1	hilfsmittel.py	21
6.2.1	experimente.py	26
7	Literaturverzeichnis	27
8	Übersicht verwendeter Hilfsmittel	28
9	Selbständigkeitserklärung	28

1 Einführung und Motivation

Eine natürliche Zahl $n > 1$ heißt Primzahl genau dann, wenn sie genau zwei Teiler besitzt, sich selbst und Eins. [7]

Der Fundamentalsatz der Arithmetik, auch Hauptsatz der elementaren Zahlentheorie genannt, besagt, dass jede ganze Zahl größer als Eins sich als Produkt von Primzahlen darstellen lässt und diese bis auf die Reihenfolge der Faktoren eindeutig ist. [11] Primzahlen bilden die Grundbausteine der natürlichen Zahlen. Jedoch ist ihre gesetzmäßige Aufeinanderfolge nicht bekannt und die Verteilung der Primzahlen unter den natürlichen Zahlen äußerst unregelmäßig. Aus diesem Grund gehören die Primzahlen zu einem interessanten Thema in der Zahlentheorie. [7]

Für die Bestimmung aller Primzahlen kleiner gleich einer vorgegebenen Zahl, existieren verschiedene Verfahren. Die Effizienz eines Algorithmus hängt dabei von ihrer Laufzeit und ihr benötigten Speicherbedarf ab. In dieser Untersuchung betrachten wir die Berechnung aller Primzahlen kleiner gleich einer gegebenen Zahl $n \in \mathbb{N}$ mithilfe verschiedener Algorithmen. Wir vergleichen dabei die Effizienz der Algorithmen, also das Verhalten ihrer Laufzeit und ihrem Speicherbedarf, abhängig von der Größe der vorgegebenen Zahl. Durch diesen Vergleich können wir ein tiefes Verständnis für die verschiedenen Algorithmen zur Primzahlberechnung gewinnen. Die Wahl eines geeigneten Algorithmus ist entscheidend für die effiziente Lösung von komplexen Problemen, die von Primzahlberechnungen abhängen. Zudem ermöglicht die Untersuchung mathematische Einblicke in die Struktur und Verteilung von Primzahlen.

Die Bestimmung aller Primzahlen bis zu einer gegebenen Zahl n ist ein klassisches Problem der Zahlentheorie. Sie findet sowohl theoretische als auch praktische Anwendungen. Dazu gehört beispielsweise die Kryptographie, die sich mit der Verschlüsselung und Sicherheit von Daten befasst. In der Praxis werden die Primzahlen in Public-Key-Verschlüsselungsverfahren wie das RSA-Kryptosystem eingesetzt. Das RSA-Kryptosystem besteht aus einem öffentlichen und einem privaten Schlüssel. Dabei ist die Wahl großer Primzahlen entscheidend für die Generierung der Schlüssel. Das Schlüsselpaar, das aus dem privaten und öffentlichen Schlüssel besteht, dient zum Entschlüsseln und Verschlüsseln von Daten, dazu gehören beispielsweise E-Mails und Chat-Nachrichten. [2]

2 Theorie

Das Wort *Primzahl* stammt von dem lat. Wort *primus* und bedeutet *der Anfang, das Erste (der Dinge)*, wobei eine Anfangszahl gemeint ist, die nicht durch andere natürliche Zahlen multiplikativ konstruiert werden kann. [5] [12]

Der Fundamentalsatz der Arithmetik besagt, dass jede ganze Zahl größer als Eins sich als Produkt von Primzahlen darstellen lässt, worauf die Primfaktorzerlegung basiert. Die Primfaktorzerlegung beschreibt die Darstellung einer positiven natürlichen Zahl n als Produkt aus Primzahlen, welche als Primfaktoren von n genannt werden. Eine Zahl n , die aus mehr als einem Primfaktor besteht, wird als zusammengesetzte Zahl genannt. [11]

Die Eigenschaft, eine Primzahl zu sein, wird auch mit *die Zahl ist prim* ausgedrückt.

[7]

Es gilt, eine natürliche Zahl $n > 1$ heißt Primzahl genau dann, wenn sie genau zwei Teiler besitzt, sich selbst und Eins. [7]

Per Definition heißt das, dass die Eins keine Primzahl ist. Der Satz von Euklid besagt, dass unendlich viele Primzahlen existieren, d.h. dass wir in der Theorie unendlich viele Primzahlen bestimmen können. Da aber noch keine Regelmäßigkeit bekannt ist, nach der Primzahlen auftreten, erschwert dies die Suche nach sehr großen Primzahlen.

Jedoch können wir alle Primzahlen kleiner gleich einer vorgegebenen Zahl bestimmen, indem wir mit der naiven Bestimmung alle Zahlen nach ihrer prim-Eigenschaft untersuchen.

2.1 Naiver Algorithmus

Der naive Algorithmus zur Bestimmung aller Primzahlen kleiner gleich $n \in \mathbb{N}$ prüft jede Zahl von 2 bis n , ob sie eine Primzahl ist.

Eine Zahl k ist prim, wenn sie nur durch Eins und sich selbst teilbar ist, d.h. sie besitzt außer diesen beiden Teilern keine weiteren.

Um zu prüfen, ob k eine Primzahl ist, reicht es aus die Teilbarkeit durch alle bekannten, kleineren Primzahlen zu prüfen. [12] Dafür betrachten wir alle natürlichen Zahlen von 2 bis $\sqrt{k}$. Wenn k durch eine natürliche Zahl q mit der Bedingung $2 \leq q \leq \sqrt{k}$ teilbar ist, d.h. $k \bmod q = 0$, dann ist k keine Primzahl. Wenn k durch keine Zahl q aus dem Intervall teilbar ist, dann ist k eine Primzahl und wird zur Liste *Primzahlen* hinzugefügt. Dieses Verfahren wird von 2 bis n durchgeführt, so dass wir letztendlich eine Liste von allen Primzahlen kleiner gleich n erhalten.

Die folgenden Beispielrechnung für $n = 6$ dient zur Veranschaulichung des naiven Algorithmus.

Beispielrechnung: $n = 6$

- $k = 2$: Prüfe Teilbarkeit durch Zahlen bis $\sqrt{2}$, 2 ist prim
- $k = 3$: Prüfe Teilbarkeit durch Zahlen bis $\sqrt{3}$, 3 ist prim
- $k = 4$: Prüfe Teilbarkeit durch Zahlen bis $\sqrt{4}$, $4 \bmod 2 = 0$, 4 ist nicht prim
- $k = 5$: Prüfe Teilbarkeit durch Zahlen bis $\sqrt{5}$, $5 \bmod 2 \neq 0$ 5 ist prim
- $k = 6$: Prüfe Teilbarkeit durch Zahlen bis $\sqrt{6}$, $6 \bmod 2 = 0$, 6 ist nicht prim

Alle Primzahlen kleiner gleich 6 lauten:

[2 3 5]

In der Beispielrechnung ist offensichtlich erkennbar, dass die geraden Zahlen 4 und 6 durch Zwei teilbar sind und somit per Definitionen keine Primzahlen sind.

Der Algorithmus ist bei großen n sehr ineffizient, da jede Zahl nach ihrer prim-Eigenschaft geprüft wird. Dabei werden viele unnötige Zahlen geprüft bei denen es klar ist, dass sie keine Primzahlen sind, wie beispielsweise alle geraden Zahlen, die größer als Zwei sind.

2.2 Sieb des Eratosthenes

Das Sieb des Eratosthenes ist ein effizienterer Algorithmus zur Bestimmung aller Primzahlen kleiner gleich einer vorgegebenen Zahl als der naive Algorithmus. Bei diesem Verfahren werden alle Zahlen bis zu einer vorgegebenen Zahl n in einer Liste betrachtet. Die Eins wird hier nicht betrachtet, da sie per Definition keine Primzahl ist. Die zunächst unmarkierten Zahlen von 2 bis n sind potenzielle Primzahlen. Die kleinste unmarkierte Zahl ist immer eine Primzahl. Wenn eine Primzahl gefunden wurde, dann werden alle ihre Vielfachen als *nicht-prim* markiert. Dann bestimmt man die nächstgrößere unmarkierte Zahl. Diese ist kein Vielfaches von den Zahlen kleiner als sie selbst, d.h. sie ist nur durch Eins und sich selbst teilbar. Nach Definition ist diese Zahl eine Primzahl und es werden wieder alle Vielfachen dieser Primzahl als *nicht-prim* markiert. Beim Markieren der Vielfachen genügt es mit dem Quadrat der Primzahl zu beginnen, da alle kleineren Vielfachen bereits markiert sind. [13]

In der Abbildung 1 wird das Verfahren von Eratosthenes für das Beispiel $n = 100$ angewendet und anschaulich dargestellt, wobei bereits alle Vielfachen der Primzahlen durchgestrichen wurden und letztendlich alle 25 Primzahlen kleiner gleich 100 in blau übrig bleiben.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Abbildung 1: Beispielanwendung für das Sieb des Eratosthenes für $n = 100$

Nach dem Algorithmus folgt, dass die Zahl 30 dreimal als *nicht-prim* markiert wird, da sie ein Vielfaches von 2, 3 und 5 ist.

Beim Sieb des Eratosthenes kommt es zu mehrfach Markierungen der zusammengesetzten Zahlen. Für sehr große n stellt das jedoch einen enorm erhöhten Rechenaufwand dar.

2.3 Sieb von Atkin

Das Sieb von Atkin ist eine optimierte Version des Sieb des Eratosthenes und wurde von A. O. L. Atkin und D. J. Bernstein entwickelt. [1] Der Algorithmus filtert potenzielle Primzahlen durch Berechnungen von quadratischen Formeln und Modulo-Rechnungen und eliminiert danach die Vielfachen der Quadrate von den Primzahlen.

Das Sieb von Atkin behandelt die Primzahlen 2 und 3 als Sonderfälle. Zu Beginn wird erstmal die vorgegebene Grenze n betrachtet. Wenn $n < 2$, dann hört der Algorithmus

auf und es existieren keine Primzahlen kleiner gleich der Grenze. Wenn $n = 2$, dann erhält man 2 als Primzahl. Bei einer Grenze n größer als 2 oder 3, fügt der Algorithmus die Zahlen 2 und 3 der Menge der Primzahlen hinzu, mit denen er beginnt. Ähnlich wie beim Sieb des Eratosthenes betrachtet der Sieb von Atkin alle Zahlen bis zu einer vorgegebenen Zahl n in einer Liste, wobei bei diesem Algorithmus alle Zahlen als *nicht-prim* gelten, bis festgestellt wird, dass sie prim sind.

Der Algorithmus hat drei Bedingungen damit eine Zahl als potenziell prim gilt. Jede Bedingung besteht aus zwei Teilen, die Berechnung von einer quadratischen Formel basierend aus x und y Werten und einer Modulo-Rechnung. Dabei sind die x und y Werte natürliche Zahlen, die im Intervall 1 und $\sqrt{n}$ liegen.

Für die erste Bedingung gilt, wenn der ausgerechnete Wert m_1 der quadratischen Gleichung $m_1 = 4x^2 + y^2$ kleiner gleich der Grenze n ist und $m_1 \bmod 12 = 1$ oder 5 ergibt, dann ist m_1 potenziell prim.

Für die zweite Bedingung gilt, wenn der ausgerechnete Wert m_2 der quadratischen Gleichung $m_2 = 3x^2 + y^2$ kleiner gleich der Grenze n ist und $m_2 \bmod 12 = 7$ ergibt, dann ist m_2 potenziell prim.

Für die dritte Bedingung gilt, wenn $x > y$, der ausgerechnete Wert m_3 der quadratischen Gleichung $m_3 = 3x^2 - y^2$ kleiner gleich der Grenze n ist und $m_3 \bmod 12 = 11$ ergibt, dann ist m_3 potenziell prim.

Alle potenziellen Primzahlen werden als *prim* markiert. Aufgrund der Struktur des Algorithmus kann es dazu kommen, dass einige zusammengesetzte Zahlen fälschlicherweise als *prim* klassifiziert werden. Um dies zu beheben, werden alle natürlichen Zahlen der zu betrachtenden Liste ab der Zahl 5 bis $\sqrt{n}$ geprüft, ob sie als *prim* gekennzeichnet wurden. Wenn eine *prim*-markierte Zahl gefunden wurde, dann werden ihr Quadrat und dessen Vielfachen wieder als *nicht-prim* zurückgesetzt. Schließlich sind in der Liste alle Primzahlen kleiner gleich n als *prim* markiert. [9] [6]

Zur Veranschaulichung des Prinzips des Siebs von Atkin dient die folgende Beispielrechnung für $n = 25$ in den Tabellen 1 und 2. In der Tabelle 1 sind alle ausgerechneten Werte der quadratischen Formeln zusammengefasst, die kleiner als die Obegrenze 25 sind. Die Tabelle 2 zeigt, ob die Überprüfung der gefundenen Werte mit den jeweiligen Modulo-Bedingungen. In Rot sind die Zahlen markiert, die ihre Modulo-Bedingung nicht erfüllen. In Grün sind die Zahlen markiert, die potenzielle Primzahlen gemäß ihrer Modulo-Bedingung sind.

Tabelle 1: Ausgerechnete Werte der quadratischen Formeln

$m_1 = 4x^2 + y^2 \leq 25$	$m_2 = 3x^2 + y^2 \leq 25$	$m_3 = x > y, 3x^2 - y^2 \leq 25$
$4 * 1^2 + 1^2 = 5$	$3 * 1^2 + 1^2 = 5$	$3 * 2^2 - 1^2 = 11$
$4 * 1^2 + 2^2 = 8$	$3 * 1^2 + 2^2 = 7$	$3 * 3^2 - 2^2 = 23$
$4 * 1^2 + 3^2 = 13$	$3 * 1^2 + 3^2 = 12$	
$4 * 1^2 + 4^2 = 20$	$3 * 1^2 + 4^2 = 19$	
$4 * 2^2 + 1^2 = 17$	$3 * 2^2 + 1^2 = 13$	
$4 * 2^2 + 2^2 = 20$	$3 * 2^2 + 2^2 = 16$	
$4 * 2^2 + 3^2 = 25$	$3 * 2^2 + 3^2 = 21$	

Tabelle 2: Überprüfung der Modulo-Bedingungen

$m_1 = 4x^2 + y^2 \leq 25$ $m_1 \bmod 12 = 1 \text{ or } 5$	$m_2 = 3x^2 + y^2 \leq 25$ $m_2 \bmod 12 = 7$	$x > y, m_3 = 3x^2 - y^2 \leq 25$ $m_3 \bmod 12 = 11$
5 mod 12 = 5	5 mod 12 $\neq$ 7	11 mod 12 = 11
8 mod 12 $\neq$ 1 or 5	7 mod 12 = 7	23 mod 12 = 11
13 mod 12 = 5	12 mod 12 $\neq$ 7	
20 mod 12 $\neq$ 1 or 5	19 mod 12 = 7	
17 mod 12 = 5	13 mod 12 $\neq$ 7	
25 mod 12 = 1	16 mod 12 $\neq$ 7	
	13 mod 12 $\neq$ 7	

Aus den beiden Tabellen ist erkennbar, dass die Zahl 25 die erste Bedingung des Algorithmus erfüllt, weshalb sie als potenzielle Primzahl gilt. Es folgt das Herausfiltern der zusammengesetzten Zahlen. Die Zahl 5 ist eine Primzahl, sodass ihr Quadrat, also $5^2 = 25$ als *nicht-prim* markiert wird. Durch das Sieb von Atkin wurden alle Primzahlen kleiner gleich 25 bestimmt:

[2 3 5 7 11 13 17 19 23]

Wie bereits im Abschnitt 2.2 erwähnt, ist das wiederholte Markieren aller Vielfachen einer Primzahl beim Sieb des Eratosthenes redundant. Dies wird beim Sieb von Atkin durch das Markieren der Quadrate und deren Vielfachen optimiert. Zudem liegt der Hauptfokus des Algorithmus bei den oben genannten Bedingungen, um potenzielle Primzahlen zu markieren, d.h. jede Zahl wird unabhängig von anderen Primzahlen betrachtet, solange sie eine der Bedingungen erfüllt.

2.4 Komplexität von Algorithmen

Die Komplexität ist ein Konzept, welches die Effizienz eines Algorithmus in Bezug auf die Laufzeit und den benötigten Speicherbedarf bewertet, wobei die Effizienz eines Algorithmus meist vom Ausgangszustand der Datenmenge abhängig ist. Die zwei Hauptaspekte der Komplexität sind die Zeitkomplexität und die Speicherkomplexität.

Die Zeitkomplexität beschreibt die Anzahl der Elementaroperationen bei einer bestimmten Eingabelänge n , d.h. sie gibt an, wie sich die Ausführungszeit des Algorithmus mit zunehmender Eingabegröße ändert. Als Elementaroperationen gelten beispielsweise arithmetische Operationen, Zuweisungen und Vergleiche, sowie auch Arrayzugriffe.

Die Speicherkomplexität gibt an, wie sich der Speicherbedarf eines Algorithmus mit zunehmender Eingabegröße verhält.

Bei der Komplexität orientiert man sich der O-Notation, am Worst Case. Für die Zeitkomplexität bedeutet das die maximale Laufzeit eines Algorithmus für eine Eingabe der Größe n . Für die Speicherkomplexität beschreibt die O-Notation den maximalen Speicherverbrauch des Algorithmus. [3] [10]

Für kleine Eingabegrößen (wie z.B. $n = 5$) ist das Messen der Laufzeit uninteressant, da die Laufzeit bei kleinen Eingaben eher durch andere Faktoren wie dem Betriebssystem

oder der Programmiersprache abhängt als durch den Algorithmus selbst. [3] Interessanter zu untersuchen ist also das Verhalten der Komplexitäten bei sehr großen Eingaben. Solche Untersuchungen geben theoretische Einblicke in die fundamentalen Grenzen eines Algorithmus. Es hilft dabei, die Unterschiede zwischen Algorithmen zu erkennen und optimalere Algorithmen zu entwickeln.

In der Tabelle 3 sind die Zeitkomplexitäten und die Speicherkomplexitäten der eingeführten Primzahl-Algorithmen zusammengefasst.

Tabelle 3: Vergleich der Zeit- und Speicherkomplexität [4][9]

Algorithmus	Zeitkomplexität	Speicherkomplexität
Naiver Algorithmus	$O(n\sqrt{n})$	$O(\frac{n}{\log n})$
Sieb des Eratosthenes	$O(n \log \log n)$	$O(n)$
Sieb von Atkin	$O(n)$	$O(n)$

Bei gleicher Eingabegröße für die drei Algorithmen gilt, dass die Zeitkomplexität des naiven Algorithmus mit $O(n\sqrt{n})$ die längste Ausführungsdauer hat. Zudem ist die Speicherkomplexität vom naiven Algorithmus am kleinsten, da $O(\frac{n}{\log n})$ die Anzahl der Primzahlen mithilfe von Logarithmen abschätzt. Die beiden anderen Algorithmen haben dabei mehr Speicherbedarf, weil sie eine Liste von Zahlen bis zu einer vorgegebenen Zahl speichern. Diese Zahlen in Python als `integer`-Datentypen gespeichert. Aufgrund der Worst-Case-Betrachtung wird für die Speicherkomplexität angenommen, dass jede Zahl als `int64`-Datentyp gespeichert wird und deswegen jede Zahl mindestens 8 Bytes verbraucht. [8] Da der Arbeitsspeicher von Computern begrenzt ist, bedeutet das für die Bestimmung aller Primzahlen kleiner gleich einem vorgegebenen Wert mithilfe vom Sieb des Eratosthenes und dem Sieb von Atkin, dass die Eingabegröße beschränkt ist. Wenn das Programm den verfügbaren Arbeitsspeicher überschreitet kann dies zu erhöhte Laufzeiten oder auch zum Absturz des Programms zur Folge haben.

Im Abschnitt 2.3 wurde erwähnt, dass das Sieb von Atkin eine optimierte Version des Sieb des Eratosthenes ist, d.h. da die Speicherkomplexität von beiden gleich ist, folgt dass die Laufzeit beim Sieb von Atkin geringer sein soll als beim Sieb des Eratosthenes.

Um zu vergleichen, ab wann der Sieb von Atkin schneller ist, werden die Laufzeiten $T_{\text{Eratosthenes}}$ und T_{Atkin} betrachtet:

$$T_{\text{Eratosthenes}} = k \cdot n \log \log n \quad T_{\text{Atkin}} = c \cdot n$$

Die Konstanten k und c hängen von der Implementierung und der Hardware ab. Zur Vereinfachung setzten wir $k = c = 1$ und es folgt der Vergleich der beiden Formeln:

Wenn $T_{\text{Atkin}}(n) < T_{\text{Eratosthenes}}(n)$:

$$n < \frac{n}{\log \log n}$$

Dies ist äquivalent zu:

$$1 < \log \log n \quad \text{bzw.} \quad n < 10^{10}$$

Daraus folgt, dass in der Theorie die Laufzeit des Siebs von Atkin erst ab einer Eingabe größer als 10^{10} schneller ist als die vom Sieb des Eratosthenes. [6]

Im folgendem Abschnitt werden wir die Zeitkomplexität sowie die Speicherkomplexität der drei Algorithmen mithilfe von Experimenten vergleichen und die Ergebnisse in Bezug zur Theorie auswerten.

3 Experimente

Laufzeit und Speicherbedarf sind wichtige Aspekte bei der Bewertung der Effizienz. Durch die Untersuchung beider Faktoren, lassen sich die Verfahren finden, die sowohl schnell als auch ressourcenschonend sind.

Bei der Implementierung der Algorithmen werden Arrays anstatt Python-Listen verwendet, da sich zeigte, dass diese einen effizienteren Umgang mit großen Datenmengen bieten. Grund dafür sind der geringere Speicherverbrauch und der schnellere Zugriff auf die Elemente. Zudem ist auch eine bessere Nutzung der NumPy-Funktionen für numerische Berechnungen vom Vorteil.

Für die Laufzeitmessung wurde eine Funktion entwickelt, welche `time.perf_counter_ns()` verwendet, um die Zeit der Ausführung in Nanosekunden zu erfassen und daraufhin in Millisekunden umwandelt. Dadurch können bereits sehr kurze Laufzeiten mit einer hohen Genauigkeit gemessen werden. Trotz hoher Präzision, führt es oft zu Schwankungen abhängig von vielen Parametern, wie Rechnerlast, Betriebssystem usw.. Um diese zu minimieren wurden für die Experimente mehrfache Messungen durchgeführt. Dabei wurde aus mehreren Durchläufen ein Mittelwert gebildet mit dem Ziel präzisere und zuverlässigere Werte zu erhalten.

Dies gilt auch für die Speicherbedarfsmessung. Dabei wird der durchschnittliche Spitzenspeicher während der Ausführung des Algorithmus ermittelt und in Mebibyte angegeben (MiB).

3.1 Laufzeitmessungen

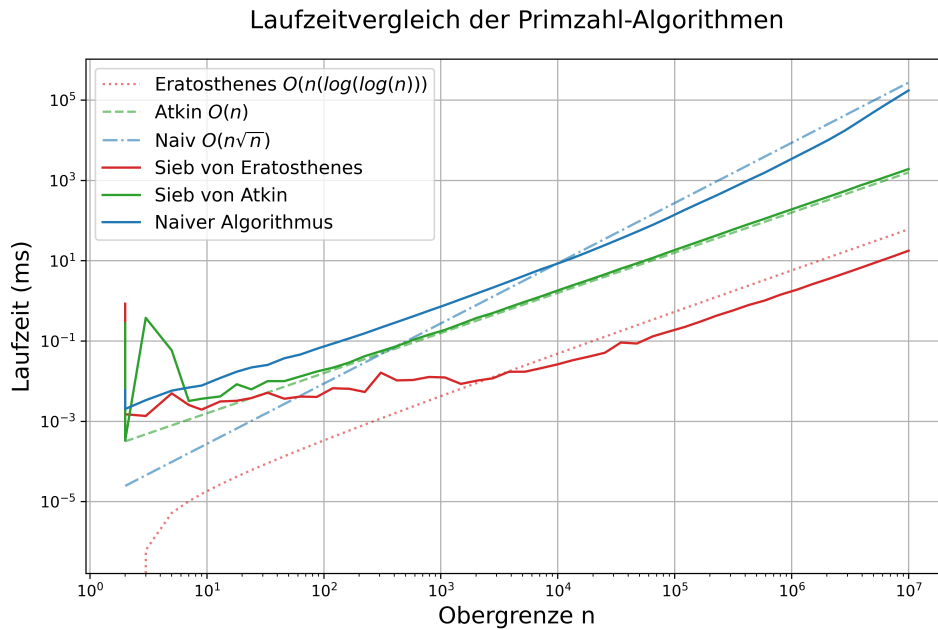


Abbildung 2: Laufzeitvergleich

Für den Vergleich des Laufzeitverhaltens der drei Algorithmen betrachten wir einen doppelt logarithmisch skalierten Plot, der die Ausführungsdauer in Millisekunden in Abhängigkeit von der Obergrenze n angibt. Um sicherzustellen, dass die Messungen repräsentativ und konsistent sind, werden die Messdaten mit vier Durchläufen jeweils 50 Datenpunkten ermittelt. Die tatsächlichen Laufzeiten der Algorithmen wurden gemeinsam mit den Referenzlinien geplottet, um die theoretischen und experimentellen Ergebnisse zu vergleichen.

Diese werden ermittelt, indem die Werte der theoretischen Komplexitäten mit dem zugehörigen Skalierungsfaktor multipliziert werden. Da in der Praxis Algorithmen verschiedene Faktoren haben, wie Hintergrundprozesse und unterschiedliche Betriebssysteme, die ihre tatsächliche Laufzeit beeinflussen, werden die theoretische Komplexitäten mit Skalierungsfaktoren multipliziert, um sie besser an die gemessenen Daten anzupassen. Für die Bestimmung des Skalierungsfaktors wird der Algorithmus für einen bestimmten Wert m ausgeführt und die tatsächliche Laufzeit gemessen. Daraufhin wird mithilfe der Laufzeitkomplexität der Referenzwert bestimmt. Der tatsächliche Messwert dividiert durch den Referenzwert, ergibt dann den Skalierungsfaktor. Es werden die Laufzeiten für 50 logarithmisch verteilte Punkte zwischen 2 und 10^7 bestimmt. Ursache ist die geplottete Laufzeitkomplexität vom Sieb von Eratosthenes, welcher aufgrund des doppelt verwendeten Logarithmus nicht für die eins definiert ist. Da bei $n = 1$, jedoch bei allen drei Algorithmen ein Array ohne Elemente zurückgegeben wird, spielt dies keine zentrale

Rolle im Vergleich der Laufzeiten.

Besonders auffällig sind die Messwerte bis 10, denn sie erscheinen willkürlich und nicht dem Rest entsprechend. Das liegt an den zu geringen Laufzeiten, welche leicht von anderen Faktoren beeinflusst werden, wie dem Betriebssystem, Hintergrundprozessen oder der Messgenauigkeit. Abgesehen davon lässt sich erkennen, dass der naive Algorithmus mit einer ansteigenden Kurve die schlechteste Leistung hat. Der Sieb von Eratosthenes zeigt deutlich bessere Ergebnisse. Die Laufzeit steigt langsamer an und nähert sich ab 10^3 seiner theoretischen Effizienz. Ähnlich dazu entsprechen auch die gemessenen Werte vom Sieb von Atkin ab ca. 10^2 seiner Referenzlinie. Dieser steigt zwar nicht so steil, wie der naive Algorithmus an, aber liegt mit seiner zeitlichen Effizienz auch bis 10^7 noch unter dem Sieb von Eratosthenes. Ab 10^8 wird das Plotten der Laufzeiten zeitintensiv.

Betrachten wir dazu die Messwerte zum Sieb von Eratosthenes und Sieb von Atkin.

$n = 10^8$

```
Sieb von Eratosthenes:  
Laufzeit (ms): 538.444208
```

```
Sieb von Atkin:  
Laufzeit (ms): 16986.699792
```

Da die Laufzeiten umgerechnet nicht länger als drei Minuten sind, lässt sich schließen dass der naive Algorithmus verantwortlich für die lange Zeit zum Plotten ist. Deswegen auch die Skalierung bis 10^7 .

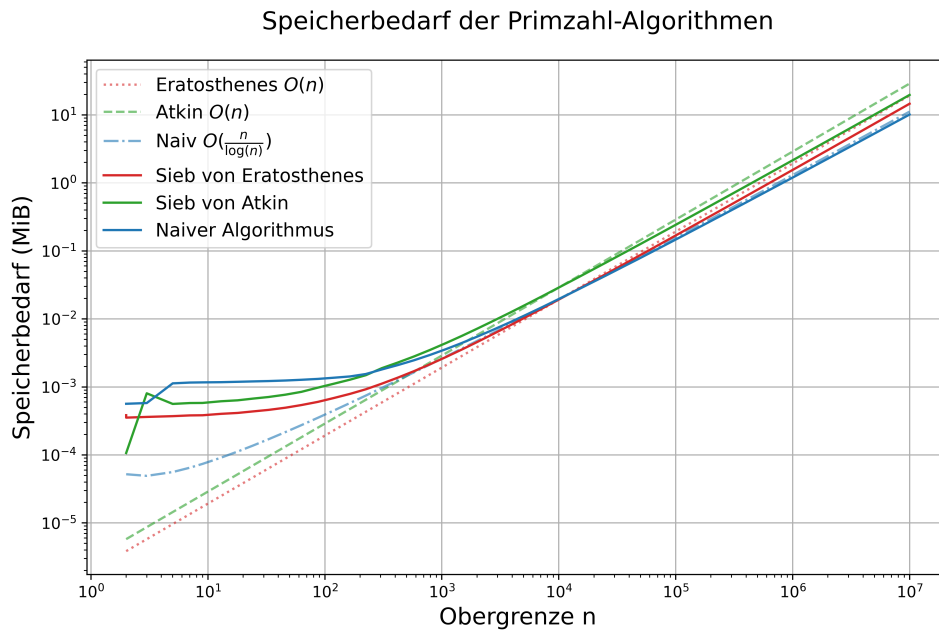


Abbildung 3: Speicherbedarfvergleich

3.2 Speicherbedarfsmessungen

Ähnlich wie bei den Laufzeitmessungen betrachten wir einen doppelt logarithmischen Plot, der den Speicherbedarf in Megabyte in Abhängigkeit von der Obergrenze n angibt. Es werden wieder mit 50 Datenpunkten, Mittelwerte aus vier Durchläufen ermittelt. Die dargestellten gestrichelten Referenzlinien, wurden mit der gleichen Methode ermittelt.

Auch beim Speicherbedarf nähern sich die Algorithmen zunehmend an die theoretischen Komplexitäten. Daraus folgt der Sieb von Eratosthenes und der Sieb von Atkin behalten eine lineare Steigung bei, wobei bis 10^7 der Sieb von Eratosthenes einen geringeren Speicherverbrauch aufweist. Der naive Algorithmus ist ab 10^4 speichereffizienter als die anderen beiden Algorithmen. Abgesehen davon weichen die Messwerte für kleine n bis etwa 10^3 stark von den Werten der theoretischen Komplexitäten ab. Die Skalierung der x-Achse gestaltet sich aus verwandten Gründen bis 10^7 . Ab 10^8 steigt die Laufzeit beim Plotten stark an.

Man betrachte dafür die Messwerte der beiden Siebe.

$n = 10^8$

Sieb von Eratosthenes:

Speicherverbrauch (MiB): 139.32421970367432

Sieb von Atkin:

Speicherverbrauch (MiB): 183.28099155426025

$$n = 10^9$$

Sieb von Eratosthenes:

Speicherverbrauch (MiB): 1341.6105890274048

Sieb von Atkin:

Speicherverbrauch (MiB): 1729.5468492507935

Es lässt sich erkennen, dass beide Algorithmen sich weiterhin linear verhalten.

4 Auswertung

Die experimentellen Ergebnisse stimmen trotz Abweichungen zu großen Teilen mit den theoretischen Erwartungen überein.

Die Untersuchungen zu den Laufzeiten entsprechen den Vorstellungen mit dem naiven Algorithmus als ineffizientestem Algorithmus und dem Sieb von Eratosthenes als schnellstem Verfahren. Solche Messwerte scheinen einleuchtend hinsichtlich der Tatsache, dass der naive Ansatz jede Zahl ohne System durchgeht, die Siebe jedoch durch Markierungen aller Vielfachen die Anzahl an Operationen minimieren. Das Sieb von Atkin zeigt sich als ineffizienter, als das Sieb von Eratosthenes. Dieser soll laut der Theorie ab ca. 10^{10} eine bessere Laufzeit erreichen. Wie sich jedoch zeigte, wächst der Speicherbedarf der beiden Siebe linear und nach Betrachtung der Messwerte lässt sich leicht rechnen, dass der Speicherbedarf vom Sieb von Atkin ab 10^{10} etwa 18.136 GB beträgt und der Speicherbedarf vom Sieb von Eratosthenes etwa 14.068 GB entspricht. Ein durchschnittlicher Computer hat bis zu 8 GB Arbeitsspeicher, wodurch er nicht reichen würde um solche Speicheranforderungen zu bewältigen.

Auch beim Speicherbedarf nähern sich die Messwerte der Referenzlinie. Hierbei hat der naive Algorithmus die besten Leistungen, da er nicht einen gesamten Array mit n Booleans speichert, sondern nur die berechneten Primzahlen.

Trotz gleicher theoretischer Speicherkomplexität der beiden Siebe, ist das Sieb von Atkin am speicherintensivsten. Denn der Algorithmus verwendet komplexere Berechnungen und zusätzliche Bedingungen zur Markierung der Zahlen als potenzielle Primzahlen. Dabei führt es zu Zwischenspeicherungen, bei denen Zahlen mehrfach markiert und überprüft werden, bevor endgültig entschieden wird, ob es sich um eine Primzahl handelt.

Obwohl die Ergebnisse in vielen Hinsichten der Theorie entsprechen, führt es teils zu Abweichungen von der Referenzlinie, besonders für kleine n . Ursache sind die bereits erwähnten zu kleinen Messwerte, welche durch viele Faktoren beeinflusst werden können, aber auch die in der Theorie nicht berücksichtigten Kosten von Iterationen in einer Schleife. Die Komplexitäten zählen zwar die Iterationen einer Schleife aber vernachlässigen Punkte, wie Konstantenfaktoren und kleinere Terme, welche sich für große Werte aufsummieren.

5 Zusammenfassung

Für die Bestimmung aller Primzahlen kleiner gleich einer vorgegebenen Zahl gibt es verschiedene Algorithmen mit denen man vorgehen kann. Problemlos kann gesagt werden,

dass der naive Algorithmus sehr einfach gestaltet ist und auch einfach zu implementieren ist. Trotz dieser Eigenschaft führen sehr große Zahlen zu hohen Laufzeiten, weshalb der Algorithmus für solche Zahlen ineffizient ist. Bezüglich der Implementierung ist das Sieb des Eratosthenes ebenfalls leicht verständlich, wobei das Sieb von Atkin wesentlich kompliziertere Bedingungen hat. In der Theorie als auch in der Praxis haben sich das Sieb des Eratosthenes und das Sieb von Atkin als effizientere Algorithmen im Bezug auf die Laufzeit erwiesen. Durch Verringern von Elementaroperationen bzw. optimierte Methoden und Ansätzen werden redundante Schritte minimiert, sodass sich das positiv auf die Laufzeiten der Algorithmen auswirkt. Die beiden Siebe sind in dem Sinne viel effizienter für die Bestimmung bei sehr großen n . In der Theorie gilt zudem, dass ab einem Wert von $n > 10^{10}$ das Sieb von Atkin eine geringere Laufzeit haben soll. In der Praxis dieser Untersuchung war es jedoch nicht möglich diese Theorie zu stützen, da der Arbeitsspeicher der genutzten Rechner für bis zu 8GB RAM begrenzt ist und der Speicherbedarf der Primzahl-Algorithmen ab solchen Werten diesen überschreitet.

In dieser Untersuchung wird hauptsächlich deutlich, dass das Sieb des Eratosthenes für die Bestimmung aller Primzahlen am effizientesten ist, da es bezüglich der Laufzeitkomplexität und der Speicherkomplexität unter den drei Algorithmen am besten abschneidet.

Interessant wäre es zu untersuchen, wie sich die Laufzeitkomplexität und die Speicherkomplexität der Algorithmen für viel größere n verhalten und insbesondere ob die Theorie des optimierten Sieb sich in der Praxis auch belegen lässt.

6 Python-Dokumentation

6.1 Bedienungsanleitung und Hauptprogramm

Das Pythonmodul zur Bestimmung aller Primzahlen kleiner gleich $n \in \mathbb{N}$ besteht aus drei Pythonprogrammen `algorithmen.py`, `hilfsmittel.py` und `experimente.py`. Das Hauptmodul `experimente.py` ist ein Experimentierskript, welches dem Nutzer ermöglicht verschiedene Algorithmen zur Bestimmung von Primzahlen kleiner gleich n zu untersuchen. Die Algorithmen werden hinsichtlich ihres Speicherbedarfs und ihrer Laufzeit anhand von gemessenen Werten verglichen. Ausgegebene Grafiken können zudem gespeichert werden. Das Experimentierskript enthält definierte Funktionen aus `algorithmen.py` und `hilfsmittel.py`.

In `algorithmen.py` sind drei Funktionen zu jeweils einem Primzahl-Algorithmus implementiert, `naiv_prim` für den naiven Algorithmus, `sieb_eratosthenes` für den Sieb des Eratosthenes und `sieb_von_atkin` für den Sieb von Atkin.

Im Programm `hilfsmittel.py` sind verschiedene Hilfsfunktionen definiert. `measure_time` dient für das Messen der Ausführungszeit eines Algorithmus und `measure_memory_usage` misst den Spitzenspeicherbedarf während der Ausführung eines Algorithmus. Das Modul enthält die Funktion `read_number`, die das Einlesen von Nutzereingaben erleichtert. Zusätzlich ist die Funktion `referenzlinie` implementiert, die Referenzlinien mithilfe der Algorithmen durch Skalierung der Referenzwerte berechnet werden. Die Funktion `plot_vergleich` in `hilfsmittel.py` erstellt einen Log-Log-Plot für die Experimente und gibt die Möglichkeit den Plot gegebenenfalls zu speichern.

Das Hauptprogramm wird nur gestartet, wenn `experimente.py` mittels `python3 experimente.py` gestartet wird.

6.1.1 Bedienungsanleitung

Starten Sie zuerst das Programm `experimente.py`. Es wird die `main()`-Funktion gestartet. Sie werden zunächst gefragt, ob Sie die Algorithmen hinsichtlich ihrer Laufzeit oder ihrem Speicherbedarf untersuchen wollen. Sie werden dazu aufgefordert "1" oder "2" einzugeben.

Wenn Sie `1` eingeben, dann haben Sie die Laufzeit gewählt und werden erneut dazu aufgefordert `1` oder `2` einzugeben, wobei bei der Eingabe `1` die Werte der gemessenen Laufzeiten des naiven Algorithmus, dem Sieb von Eratosthenes und Sieb von Atkin für eine selbstgewählte Obergrenze zurückgegeben wird. Bei der Eingabe `2` werden die Laufzeiten der Algorithmen durch eine ausgewählte Obergrenze grafisch wiedergegeben, die im Nachhinein durch die Eingabe `1` gespeichert werden kann. Wenn der Plot nicht gespeichert werden soll, dann muss `2` eingegeben werden. Für beide Optionen soll eine Obergrenze zwischen 2 und 10^6 gewählt werden.

Wenn Sie nun zu Beginn nicht `1` sondern `2` eingegeben haben, dann haben Sie die Untersuchung für den Speicherbedarf gewählt und werden erneut dazu aufgefordert `1` oder `2` einzugeben, wobei bei der Eingabe `1` die Werte des gemessenen Speicherverbrauchs der Algorithmen für eine selbstgewählte Obergrenze wiedergegeben wird. Bei der Eingabe `2` werden die gemessenen Werte für eine ausgewählte Obergrenze grafisch dargestellt und

können wie oben durch die Optionen *1* und *2* entweder gespeichert werden oder auch nicht. Für beide Optionen soll eine Obergrenze zwischen 2 und 10^5 gewählt werden.

Bei einer leeren Eingabe wird das Programm abgebrochen. Bei einer Eingabe außerhalb der gegebenen Optionen, d.h. bei einem falschen Datentyp, wie beispielsweise einem `string` oder außerhalb der gegebenen Intervall für die Obergrenze, wird der Nutzer dazu aufgefordert eine erneute Eingabe zu betätigen.

Beim Speichern der Grafiken wird der Plot für die Laufzeit im gleichen Ordner wie `experimente.py` als `laufzeit_plot.png` gespeichert. Beim Speichern der Grafik des Speicherbedarfs wird der Plot ebenfalls im gleichen Ordner wie `experimente.py` als `speicherbedarf_plot.png` gespeichert. Zu berücksichtigen ist, dass keine png-Datei mit dem gleichen Dateinamen im Order des Programms vorliegen sollte, da diese sonst überschrieben wird.

6.1.2 Beispielausführung des Hauptprogramms

Zur Veranschaulichung des Programms führen wir zwei Beispielausführungen durch.

Beim Starten des Programms wird dem Nutzer folgendes angezeigt:

```
Ihnen stehen 3 verschiedene Algorithmen zur Verfügung für die Bestimmung
aller Primzahlen bis zur gewählten Obergrenze n.
```

```
Und zwar ein naiver Algorithmus, der Sieb von Eratosthenes und
der Sieb von Atkin.
```

```
Dieses Programm dient zur Untersuchung dieser Algorithmen.
```

```
Was wollen sie zunächst untersuchen?
```

1. Die Algorithmen hinsichtlich ihrer Laufzeiten
 2. Die Algorithmen hinsichtlich ihrem Speicherbedarf
- ```
Ihre Wahl (1 oder 2):
```

Bei der Eingabe *1* liefert dies folgende Ausgabe:

```
Ihre Wahl (1 oder 2): 1
```

```
Wollen Sie...
```

1. Die Laufzeiten für eine gewählte Obergrenze bestimmen
2. Die Laufzeiten grafisch vergleichen

```
Ihre Wahl (1 oder 2):
```

Hier zeigen wir die zwei Beispielausführungen, wähle der Nutzer zunächst *1* und danach die Obergrenze 1000000, dann folgt die Ausgabe:

```
Ihre Wahl (1 oder 2): 1
```

```
Wählen Sie eine Obergrenze (zwischen 1 und 10^6): 1000000
```

```
Naiver Algorithmus:
```

```
Laufzeit (ms): 3519.276458
```



Sieb von Eratosthenes:  
Laufzeit (ms): 1.844625

Sieb von Atkin:  
Laufzeit (ms): 152.095042

Wähle der Nutzer, anstatt der vorherigen 1, nun die 2 und ebenfalls die Obergrenze 1000000. Zudem gebe der Nutzer 1 zum Speichern des folgenden Plots ein, dann folgt die Ausgabe:

Ihre Wahl (1 oder 2): 2

Wählen Sie eine Obergrenze (zwischen 1 und  $10^6$ ): 1000000

Wollen Sie den Plot unter dem Namen laufzeit\_plot.png speichern?

Es sollte keine Datei mit dem gleichen Dateiname im Ordner des Programms vorliegen, da diese sonst überschrieben wird.

(1 für ja, 2 für nein): 1

Plot gespeichert als laufzeit\_plot.png.

Der Plot wird als laufzeit\_plot.png im selben Ordner wie das Programm gespeichert:

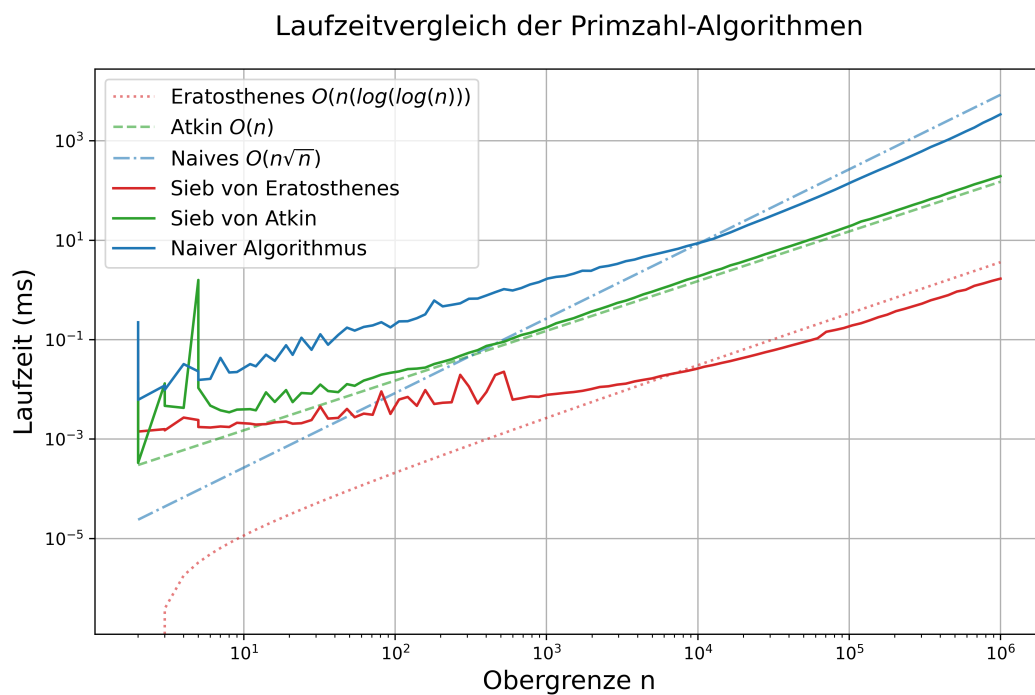


Abbildung 4: Beispielpplot für  $n = 10^6$

## 6.2 Schnittstellendokumentation

Das Hauptmodul enthält Funktionen aus den Programmen `algorithmen.py` und `hilfsmittel.py`. In `algorithmen.py` sind drei verschiedene Funktionen zur Primzahlbestimmung bis zu einer Grenze definiert, In `hilfsmittel.py` sind Hilfsfunktionen wie das Messen der Laufzeit oder das Messen des Spitzen-Speicherbedarfs für das Experimentierskript vordefiniert.

*Bemerkung 1:* Die Reihenfolge der Eingabeparameter für die Funktionen wurden unter **Parameters** von oben nach unten aufgelistet.

*Bemerkung 2:* Da in jeder Pythondatei eine `main()` Funktion vorhanden ist, wird vor jeder `main()` Funktion der jeweilige Dateiname dazugeschrieben, um Verwechslungen zu vermeiden.

*Bemerkung 3:* Mit **Input** ist die von der Funktion aufgeforderten Eingabe gemeint.

### 6.2.1 `algorithmen.py`

Das Programm `algorithmen.py` enthält die drei Funktionen `naiv_prim`, `sieb_eratosthenes` und `sieb_von_atkin` und zusätzlich eine `main()` Funktion.

#### **`naiv_prim()`**

Diese Funktion bestimmt alle Primzahlen kleiner gleich der einzugebenden Zahl  $n$  mithilfe der naiven Bestimmung.

**Parameters:**

$n$  (*int*) Obergrenze der zu berechnenden Primzahl

**Returns:**

(*np.array*) Array mit allen Primzahlen kleinergleich  $n$

In der Abbildung 5 ist das Flussbild der `naiv_prim` Funktion abgebildet, welche den Verlauf der Funktion veranschaulicht.

#### **`sieb_eratosthenes()`**

Diese Funktion bestimmt alle Primzahlen kleiner gleich der einzugebenden Zahl  $n$  mithilfe des Primzahl-Algorithmus Sieb des Eratosthenes.

**Parameters:**

$n$  (*int*) Obergrenze der zu berechnenden Primzahl

**Returns:**

(*np.array*) Array mit allen Primzahlen kleinergleich  $n$

In der Abbildung 6 ist das Flussbild der `sieb_eratosthenes` Funktion dargestellt, welche den Verlauf der Funktion zeigt.

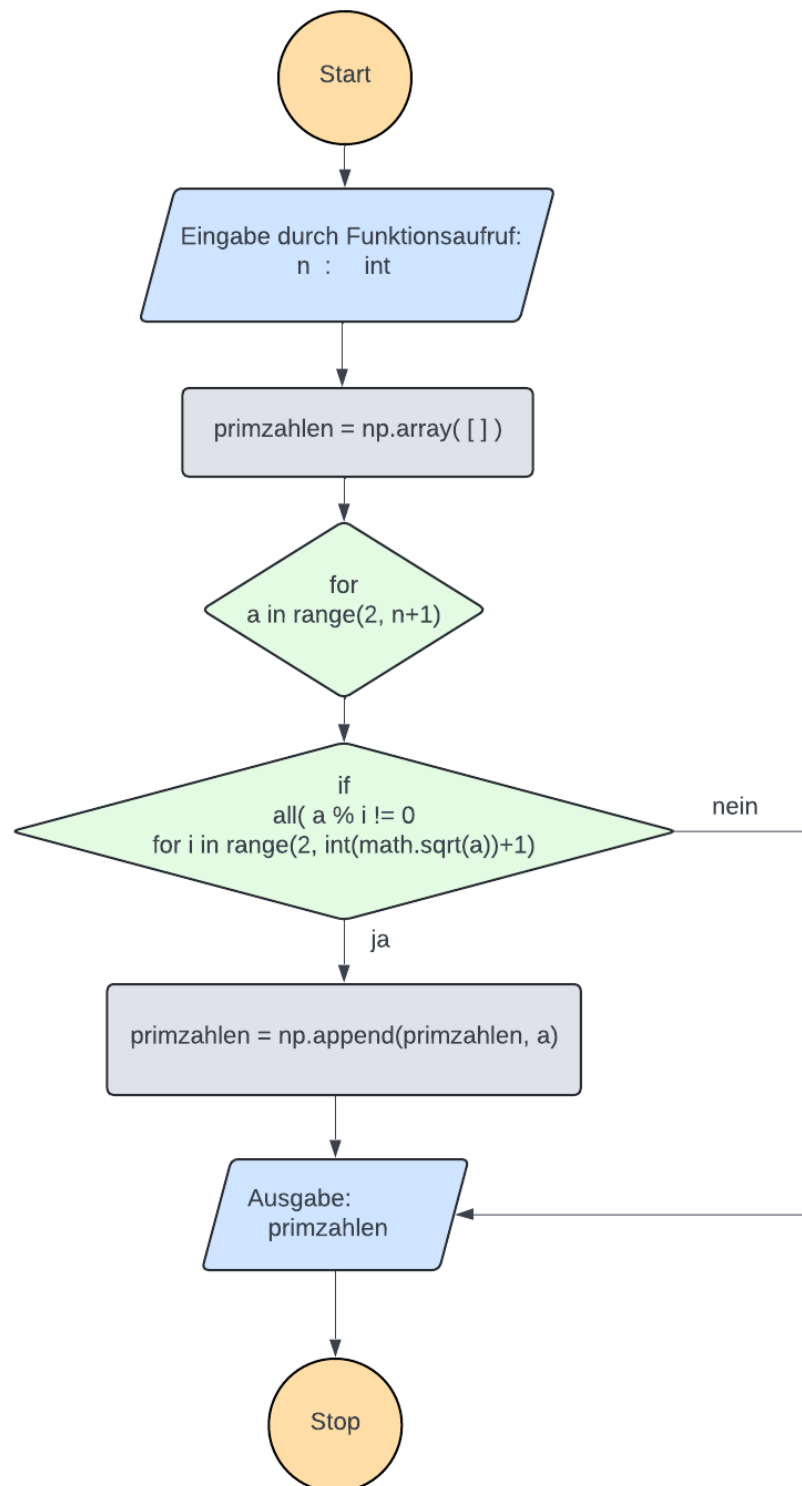


Abbildung 5: Flussbild der `naiv_prim` Funktion

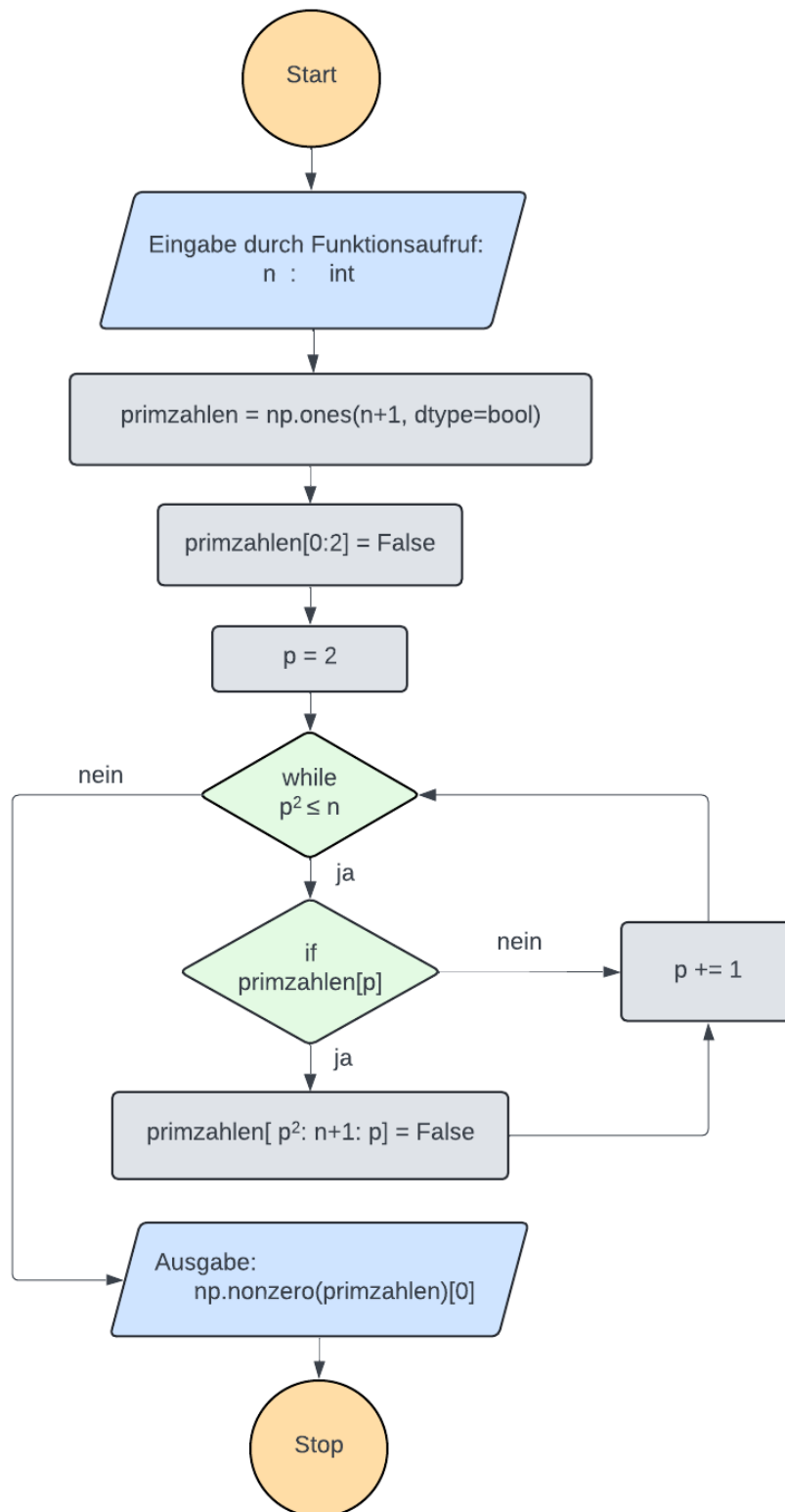


Abbildung 6: Flussbild der `sieb_eratosthenes` Funktion

### **sieb\_von\_atkin()**

Diese Funktion bestimmt alle Primzahlen kleiner gleich der einzugebenden Zahl  $n$  mithilfe des Siebs von Atkin.

**Parameters:**

$n$  (*int*) Obergrenze der zu berechnenden Primzahl

**Returns:**

(*np.array*) Array mit allen Primzahlen kleiner gleich  $n$

In der Abbildung 7 ist das Flussbild der `sieb_von_atkin` Funktion, welche den Verlauf der Funktion anschaulich darstellt.

### **algorithmen.main()**

Diese Funktion ist die `main()` Funktion von `algorithmen.py` und gibt mithilfe der Funktionen `naiv_prim`, `sieb_eratosthenes` und `sieb_von_atkin` für jeden Algorithmus alle Primzahlen kleiner gleich der einzugebenen Zahl  $n$  an.

**Parameters:** *None*

**Input:**

$n$  (*int*) Obergrenze der zu berechnenden Primzahl

**Exceptions:**

(*ValueError*) wenn die angegebene Obergrenze kleiner als 0 ist oder der falsche Datentyp eingegeben wurde, dann wird der Nutzer dazu aufgefordert eine erneute Eingabe zu betätigen

**Returns:** *None*, denn diese Funktion gibt keine Werte zurück, sondern gibt die Ergebnisse der Berechnungen direkt in der Konsole aus.

### **6.2.1 hilfsmittel.py**

Das Modul besteht aus 9 Funktionen und ihrer `main()` Funktion.

### **measure\_time()**

Diese Funktion misst die Laufzeit eines Algorithmus.

**Parameters:**

`algorithm` (*function*) Algorithmus, dessen Laufzeit gemessen werden soll

$n$  (*int*) Parameter, der an den Algorithmus übergeben wird

`runs` (*int*) Anzahl der Messungen, die durchgeführt werden soll um genauere Messwerte zu erhalten, Standardwert: 1

**Returns:**

(*float*) die gemessene Laufzeit des Algorithmus in Millisekunden

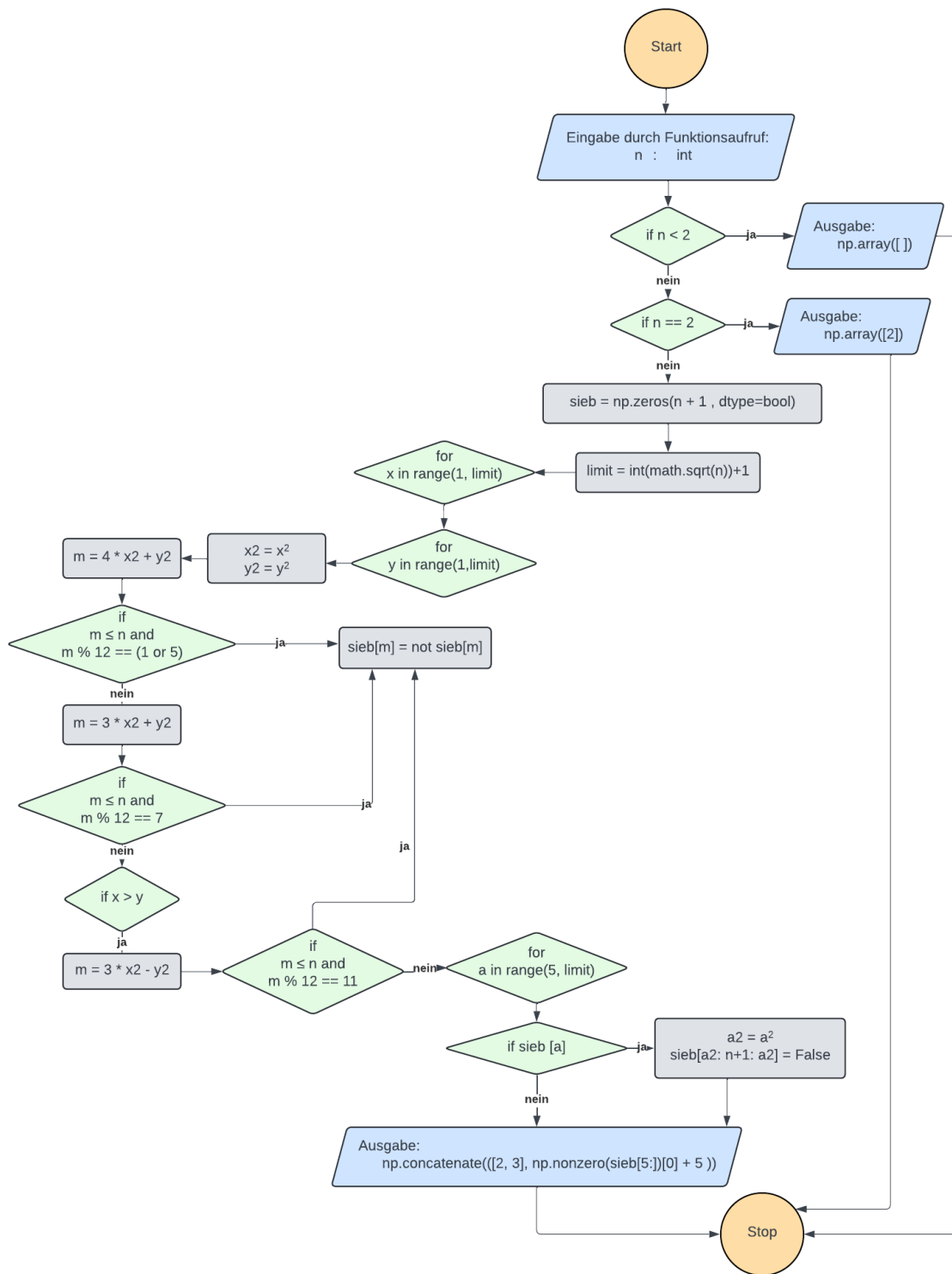


Abbildung 7: Flussbild der `sieb_von_atkin` Funktion

### **measure\_memory\_usage()**

Diese Funktion misst den Spitzen-Speicherbedarf während der Ausführung des Algorithmus.

#### **Parameters:**

**algorithm** (*function*) Algorithmus, dessen Speicherbedarf gemessen werden soll  
**n** (*int*) Parameter, der an den Algorithmus übergeben wird  
**runs** (*int*) Anzahl der Messungen, die durchgeführt werden soll um genauere Messwerte zu erhalten, Standardwert: 1

#### **Returns:**

(*float*) der gemessene Spitzen-Speicherbedarf des Algorithmus in Mebibyte

### **read\_number()**

Diese Funktion liest eine einzugebene Zahl x ein und prüft die Nutzereingabe auf ihre Gültigkeit.

#### **Parameters:**

**question** (*str*) Frage, die der Nutzer beantworten soll  
**lower\_limit** (*float*) untere Intervallgrenze, Standardwert: -Inf  
**upper\_limit** (*float*) obere Intervallgrenze, Standardwert: Inf  
**data\_type** (*type*) Zahlentyp der Zahl, also entweder **float** oder **integer**

#### **Input:**

**x** (*int*) oder *float* der einzugebene Wert, der nach seiner Gültigkeit geprüft wird

#### **Exceptions:**

(*ValueError*) wenn die Nutzereingabe x den falschen Datentyp hat oder wenn die Eingabe leer ist, dann wird das Programm abgebrochen ohne eine Fehlermeldung anzuzeigen, die Exception wird durch *raise Exception* erzeugt

#### **Returns:** *None*, wenn die Nutzereingabe der Zahl leer ist (Abbruch des Programms)

(*int*) oder (*float*) je nach Eingabeparameter von **data\_type**, also die eingegebene Zahl x wird im Datentyp **data\_type** ausgegeben

In der Abbildung 8 ist das Flussbild der **read\_number** Funktion abgebildet, die den Verlauf der Funktion veranschaulicht.

### **referenzlinie()**

Diese Funktion berechnet die Referenzlinie durch Skalierung eines Referenzwertes.

#### **Parameters:**

**untersuchungsfunktion** (*function*) Funktion, die abhängig von **algorithmus** und **m** einen Wert berechnet  
**algorithm** (*function*) Algorithmus, der untersucht wird  
**m** (*int*) Parameter, anhand welchem der Referenzwert gebildet wird  
**x** (*array*) x-Werte, der Messungen

#### **Returns:**

(*numpy.ndarray*) y-Werte der Referenzlinie

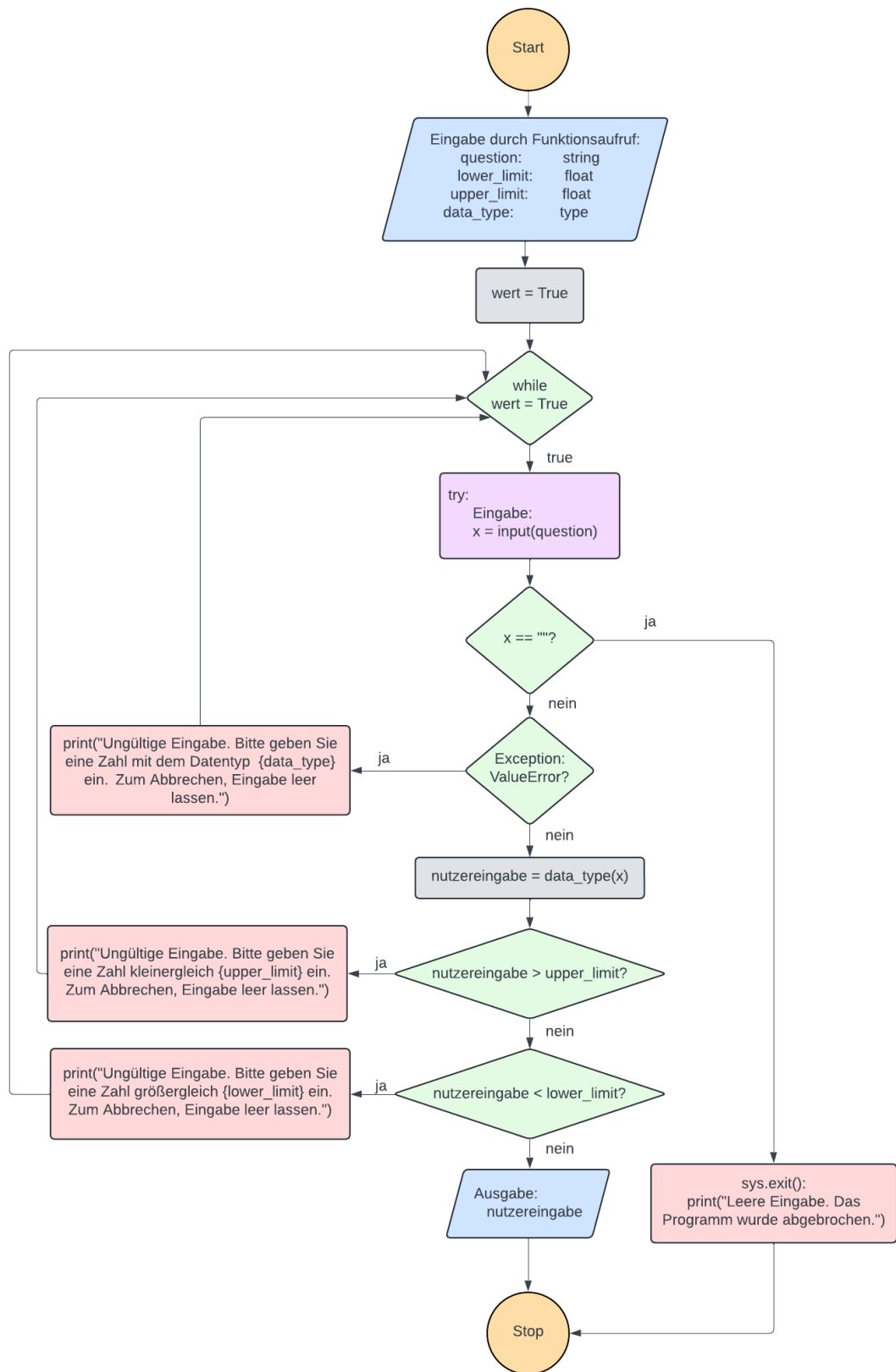


Abbildung 8: Flussbild der `read_number` Funktion



### **plot\_vergleich()**

Diese Funktion erzeugt einen Log-Log-Plot für den Vergleich von Daten mit optionalen Skalierungslinien.

#### **Parameters:**

**x** (*numpy.ndarray*) die zu plottenden Daten, x-Werte  
**y** (*list of Lists*) Liste von y-Werten für die darzustellenden Funktionen  
**labels** (*list of str*) Liste von Labels  
**title** (*str*) Titel des Plots  
**ylabel** (*str*) Bezeichnung der y-Achse  
**y\_scaling** (*list of lists*) Liste von y-Werten für die Referenzlinien, dieser Parameter ist optional, Standardwert = None  
**labels\_scaling** (*list of str*) Liste von Labels für die Referenzlinien, dieser Parameter ist optional, Standardwert = None  
**xlabel** (*str*) Bezeichnung der x-Achse, Standardwert = 'Obergrenze n'  
**filename** (*str*) Dateiname zum Speichern des Plots, dieser Parameter ist optional, Standardwert = None

**Returns:** *None*, diese Funktion gibt keine Werte zurück, sondern erzeugt und zeigt den Plot an bzw. speichert ihn in einer png-Datei ein

### **reference\_n()**

Diese Funktion berechnet Werte für die Komplexität  $O(n)$ .

#### **Parameters:**

**x** (*array*) Eingabewerte

#### **Returns:**

(*array*) Referenzwerte  $x$

### **reference\_n\_log()**

Diese Funktion berechnet Werte für die Komplexität  $O(n \log \log n)$ .

#### **Parameters:**

**x** (*array*) Eingabewerte

#### **Returns:**

(*array*) Referenzwerte  $x \cdot \log \log x$

### **reference\_n\_wurzel()**

Diese Funktion berechnet Werte für die Komplexität  $O(n\sqrt{n})$ .

#### **Parameters:**

**x** (*array*) Eingabewerte

#### **Returns:**

(*array*) Referenzwerte  $x \cdot \sqrt{x}$

### **reference\_n\_logn()**

Diese Funktion berechnet Werte für die Komplexität  $O(n \frac{n}{\log n})$ .

**Parameters:**

*x* (*array*) Eingabewerte

**Returns:**

(*array*) Referenzwerte  $\frac{x}{\log x}$

### **hilfsmittel.main()**

Diese Funktion ist das Hauptprogramm von `hilfsmittel.py` und dient zum Aufrufen der obigen eingeführten Funktionen aus dem Programm. Die Funktion gibt nach einer gültigen Eingabe die gemessenen Laufzeiten und den Speicherverbrauch der drei Primzahl-Algorithmen aus und plottet zusätzlich das Laufzeitverhalten, wobei der Plot gespeichert werden kann.

**Parameters:** *None*

**Exceptions:**

(*ValueError*) wenn der falsche Datentyp eingegeben wurde oder wenn die Eingabe leer ist, dann wird das Programm abgebrochen ohne eine Fehlermeldung anzuzeigen, die Exception wird durch *raise Exception* erzeugt

**Returns:** *None*, denn diese Funktion gibt keine Werte zurück, sondern gibt die Ergebnisse der Berechnungen und den Plot direkt in der Konsole aus.

### **6.2.1 experimente.py**

`experimente.py` ist ein Experimentierskript, welches dem Nutzer ermöglicht verschiedene Algorithmen zur Bestimmung von Primzahlen kleiner gleich *n*, hinsichtlich ihres Speicherbedarfs oder ihres Laufzeitverhalten zuvergleichen, indem nur die ausgerechneten Ergebnisse oder diese dessen Plot bis wiedergegeben wird.

### **experimente.main()**

Diese Funktion ist das Hauptprogramm des Experimentierskripts.

**Parameters:** *None*

**Input:**

'1' oder '2' (*int*) Möglichkeiten, die dem Nutzer gegeben werden, was er tun möchte  
*n* (*int*) Obergrenze der zu berechnenden Primzahl

**Exceptions:**

(*ValueError*) wenn der falsche Datentyp eingegeben wird oder die Eingabe leer ist, dann wird das Programm abgebrochen ohne eine Fehlermeldung anzuzeigen, die Exception wird durch *raise Exception* erzeugt

**Returns:** *None*, diese Funktion gibt keine Werte zurück, sondern gibt die Ergebnisse der Berechnungen direkt in der Konsole aus oder erzeugt und zeigt den Plot an bzw. speichert ihn einer png-Datei ein.

## 7 Literaturverzeichnis

### Literatur

- [1] A. O. L. ATKIN, D. J. BERNSTEIN: *PRIME SIEVES USING BINARY QUADRATIC FORMS*. <https://www.ams.org/journals/mcom/2004-73-246/S0025-5718-03-01501-1/S0025-5718-03-01501-1.pdf>, 2003. Zugriff am 24. Juni 2024.
- [2] ARIANIT MIFTARIE, FELIX ELSER, JOHANNA RIEDEL: *Primzahlen*. <https://funfacts.mathi.uni-heidelberg.de/index.php/Primzahlen>, 2021. Zugriff am 29. Juni 2024.
- [3] BÖHM, CHRISTIAN: *Komplexität von Algorithmen*. <https://www.dbs.ifi.lmu.de/Lehre/NFInfoSW/WS0708/Skript/Folien09.pdf>, 2007. Zugriff am 24. Juni 2024.
- [4] GEEKSFORGEEKS: *How is the time complexity of Sieve of Eratosthenes is  $O(n \log \log n)$ ?* <https://www.geeksforgeeks.org/how-is-the-time-complexity-of-sieve-of-eratosthenes-is-nloglogn/>, 2023. Zugriff am 24. Juni 2024.
- [5] GEORGES, KARL ERNST: *Ausführliches lateinisch-deutsches Handwörterbuch*. <http://www.zeno.org/Georges-1913/A/prior>, 1913. Zugriff am 28. Juni 2024.
- [6] ICHI.PRO: *Sieb von Atkin - Die theoretische Optimierung der Primzahlengenerierung*. <https://ichi.pro/de/sieb-von-atkin-die-theoretische-optimierung-der-primzahlengenerierung-251174113910312>. Zugriff am 24. Juni 2024.
- [7] MATHEMATIKALPHA: *Primzahlen und Primzahltests*. <https://mathematikalpha.de/wp-content/uploads/2016/12/03-Primzahlen.pdf>, 2016. Zugriff am 24. Juni 2024.
- [8] NUMPY DEVELOPERS: *NumPy: Understanding Data Types*. <https://numpy.org/doc/stable/user/basics.types.html>, 2024. Zugriff am 04. Juli 2024.
- [9] RATHORE, ANUJ: *Atkin-Sieb*. <https://www.geeksforgeeks.org/sieve-of-atkin/>, 2023. Zugriff am 24. Juni 2024.
- [10] WEGNER, FRANK: *Materialien zum Unterricht - Komplexität von Algorithmen*. <https://mrge.de/lehrer/wegner/python/komplex.php>, 2024. Zugriff am 24. Juni 2024.
- [11] WIKIPEDIA: *Primfaktorzerlegung* — *Wikipedia, die freie Enzyklopädie*. <https://de.wikipedia.org/w/index.php?title=Primfaktorzerlegung&oldid=229103424>, 2022. Zugriff am 28. Juni 2024.
- [12] WIKIPEDIA: *Primzahl* — *Wikipedia, die freie Enzyklopädie*. <https://de.wikipedia.org/wiki/Primzahl>, 2024. Zugriff am 24. Juni 2024.

- [13] WIKIPEDIA: *Sieb des Eratosthenes* — *Wikipedia, die freie Enzyklopädie*. [https://de.wikipedia.org/w/index.php?title=Sieb\\_des\\_Eratosthenes&oldid=245457930](https://de.wikipedia.org/w/index.php?title=Sieb_des_Eratosthenes&oldid=245457930), 2024. Zugriff am 24. Juni 2024.

## 8 Übersicht verwendeter Hilfsmittel

- <https://overleaf.com>: für die Erstellung der  $\text{\LaTeX}$  Dokumente: Bericht und Python Dokumentation
- Lucidchart: Erstellung der Flussbilder der Funktion in der Schnittstellendokumentation, <https://lucid.app/lucidchart/>

## 9 Selbständigkeitserklärung

Wir versichern, dass wir in dieser schriftlichen Studienarbeit alle von anderen Autor:innen wörtlich übernommenen Stellen wie auch die sich an die Gedankengänge anderer Autoren:innen eng anlehnenden Ausführungen unserer Arbeit besonders gekennzeichnet und die entsprechenden Quellen angegeben habe. Zusätzlich führen wir den Einsatz von IT-/KI-gestützten Schreibwerkzeugen zur Anfertigung dieser Arbeit im Abschnitt „Übersicht verwendeter Hilfsmittel“ im Anhang des eingereichten pdf-Dokumentes vollständig auf. Hierin haben wir die Verwendung solcher Tools vollständig durch Angabe ihres Produktnamens, unserer Bezugsquelle (z.B. URL) und Angaben zu genutzten Funktionen der Software sowie zum Nutzungsumfang dokumentiert. Bei der Erstellung dieser Studienarbeit haben wir durchgehend eigenständig und beim Einsatz IT-/KI-gestützter Schreibwerkzeuge steuernd gearbeitet.